

SPIP: Change Data Capture (CDC) Support

Author: Gengliang Wang

Created: 2026-02-24

JIRA: <https://issues.apache.org/jira/browse/SPARK-55668>

Q1. What are you trying to do? Articulate your objectives using absolutely no jargon.

Today, if you want to see what rows changed in a table — which rows were added, updated, or deleted — between two points in time, each table format (Delta Lake, Iceberg, Hudi) forces you to learn its own unique syntax and APIs. There is no standard way in Spark to ask "show me what changed." This capability is commonly known as Change Data Feed (CDF) or Change Data Capture (CDC).

We want to add a simple, built-in way to query row-level changes from any supported table, using a single consistent SQL syntax and DataFrame API. A user should be able to write:

SQL

```
-- Batch: get changes between two versions
SELECT * FROM my_table CHANGES FROM VERSION 10 TO VERSION 20;

-- Streaming: continuously process changes
CREATE STREAMING TABLE cdc_sink AS
SELECT * FROM STREAM my_table CHANGES FROM VERSION 0;
```

and get back the rows that were inserted, updated, or deleted — regardless of whether the table is stored in Delta Lake, Iceberg, or any other format. This should work for both batch reads and streaming pipelines.

Q2. What problem is this proposal NOT designed to solve?

- **Replacing existing connector-specific CDC features.** Delta Lake's `table_changes()`, Iceberg's `.changes` virtual table, and similar connector-specific APIs will continue to work. This proposal provides a standardized alternative, not a forced migration.
- **Schema evolution across versions.** Handling schema differences (added/removed/renamed columns) across the queried version range is the connector's responsibility. Spark provides the framework; connectors handle the data transformation.

- **Cross-table change tracking or CDC joins.** This proposal addresses changes within a single table. Correlating changes across multiple tables (e.g., joining the change feed of two tables) is possible using standard SQL on top of CDC results but is not a first-class feature.
- **Built-in catalog and file formats.** This feature is designed for Data Source V2 connectors. None of Spark's built-in catalogs or file formats (Parquet, ORC, JSON, CSV, etc.) will support it. The target audience is V2 connectors such as Delta Lake, Iceberg, and Hudi that already maintain the versioning and change-tracking metadata required to serve change data.

Q3. How is it done today, and what are the limits of current practice?

Today, each connector implements CDC independently with incompatible APIs:

Connector	SQL Syntax	DataFrame API
Delta Lake	<code>SELECT * FROM table_changes('t', 10, 20)</code>	<code>.option("readChangeFeed", "true").table("t")</code>
Iceberg	<code>SELECT * FROM t.changes</code>	<code>.format("iceberg").load("t.changes")</code>
Hudi	N/A (custom MOR read)	<code>.option("hoodie.datasource.query.type", "incremental").load(path)</code>

Limitations of the current approach:

1. **No portability.** Switching table formats requires rewriting all CDC queries.
2. **Inconsistent semantics.** Each connector defines its own metadata columns (Delta: `_change_type`, Iceberg: `_change_type`, but with different values), deduplication behavior, and update representation. Users must learn each format's quirks.
3. **Duplicated effort with no Spark-level optimizations.** Every connector independently implements the same post-processing logic: removing carry-over rows from copy-on-write operations, detecting updates from insert/delete pairs, and computing net changes. This is error-prone, wastes engineering effort, and means Spark's optimizer cannot reason about or improve CDC queries.

Q4. What is new in your approach and why do you think it will be successful?

1. Dedicated SQL Syntax with the **CHANGES** Keyword

We introduce **CHANGES FROM ... TO ...** as first-class SQL syntax, clearly distinguishing CDC queries from time-travel snapshots:

```
SQL
-- Time travel (existing): returns the table as it looked at version 5
SELECT * FROM table VERSION AS OF 5;

-- CDC (new): returns what changed between version 5 and version 10
SELECT * FROM table CHANGES FROM VERSION 5 TO VERSION 10;
```

This eliminates ambiguity and makes CDC intent explicit. The syntax supports version-based ranges, timestamp-based ranges, per-bound **INCLUSIVE/EXCLUSIVE** keywords, and optional processing options via a **WITH** clause.

2. Minimal Connector Interface (**Changelog**)

Instead of requiring connectors to implement a complex API, we define a minimal **Changelog** interface. Connectors only need to:

- Declare their columns (including standardized metadata columns)
- Provide a **ScanBuilder** for reading raw change data
- Declare properties of their change data (e.g., whether it contains carry-over rows)

Spark handles the rest — wrapping the **Changelog** in an internal **ChangelogTable**, injecting update detection, carry-over removal, and net change computation as analyzer rules.

3. Engine-Level Post-Processing

Spark takes responsibility for common CDC post-processing that every connector would otherwise implement independently:

- **Carry-over removal:** Filtering out identical insert/delete pairs produced by copy-on-write file rewrites
- **Update detection:** Deriving **update_preimage/update_postimage** from raw insert/delete pairs using window functions
- **Net change computation:** Collapsing multiple changes to the same row into the net effect

This approach pushes complexity into Spark's optimizer where it can be implemented once, tested thoroughly, and benefit from future Spark optimizations (e.g., columnar execution, AQE).

Why we believe this will succeed:

- The design imposes minimal burden on connector developers (implement one interface, a few boolean methods, and a scan builder)
- It reuses existing DSV2 infrastructure (no logical plan changes needed)
- It provides immediate value: users get a portable, standardized API; connectors get free post-processing logic

Q5. Who cares? If you are successful, what difference will it make?

Data Engineers will be able to write portable CDC queries for ETL/ELT pipelines, data replication, and audit logging that work across Delta Lake, Iceberg, and Hudi without modification — eliminating table-format lock-in.

Connector Developers will implement a minimal **Changelog** interface and get carry-over removal, update detection, and net change computation for free from the Spark engine, reducing duplicated effort across the ecosystem.

The Spark Ecosystem benefits from a unified CDC story. CDC is fundamental to data warehousing, real-time analytics, and incremental processing. Standardizing it strengthens Spark's position as the universal data processing engine.

Q6. What are the risks?

1. **Connector adoption lag.** The value of this proposal depends on connectors implementing the **Changelog** interface. If major connectors (Delta Lake, Iceberg) are slow to adopt, the feature will see limited use. *Mitigation:* We will work with connector maintainers early and provide reference implementations.
2. **Performance regression for deduplication.** Spark-computed carry-over removal and net change computation use window functions, which may be slower than optimized connector-specific implementations for some workloads. *Mitigation:* If window functions prove to be a bottleneck from benchmarks, we will pivot to alternative strategies(e.g. Iceberg's custom iterator).
3. **Schema evolution complexity.** CDC queries spanning many versions may encounter schema changes. Since schema evolution is the connector's responsibility, different connectors may handle this inconsistently. *Mitigation:* The DSV2 Changelog interface is intentionally designed to delegate schema evolution decisions to connectors — each connector can decide how to resolve schema changes (e.g.,

applying the latest schema, failing on incompatible changes) based on the semantics of its underlying format.

Q7. How long will it take?

The implementation targets **Spark 4.2** with an estimated effort of 2-3 months.

Q8. What are the mid-term and final "exams" to check for success?

Mid-term — Phase 1 (API Changes):

- `Changelog` connector interface, `ChangelogInfo`, `ChangelogTable` internal wrapper, and `TableCatalog.loadChangelog()` extension are implemented and stable
- `CHANGES` SQL syntax (batch and streaming) is parseable, resolvable, and executable
- `DataFrameReader.changes()` and `DataStreamReader.changes()` APIs work end-to-end

Final — Phase 2 (Analyzer Rules):

- `ResolveChangelogTable` rule implements carry-over removal, update detection, and net change computation
- Filter pushdown restrictions ensure CDC correctness
- All deduplication modes (`none`, `dropCarryovers`, `netChanges`) produce correct results
- Documentation is complete: SQL reference, DataFrame API reference, and connector developer guide
- No regressions in existing time-travel or table-reading functionality

Appendix A. Proposed API Changes

A.1 SQL Syntax Changes

Batch Query

```
SQL
SELECT select_list
FROM table_name cdc_clause [ WITH ( option = value [, ...] ) ]
[ WHERE condition ]
[ other_clauses ]
```

```
cdc_clause:
  CHANGES FROM VERSION starting_version [ INCLUSIVE | EXCLUSIVE ]
    [ TO VERSION ending_version [ INCLUSIVE | EXCLUSIVE ] ]
  CHANGES FROM TIMESTAMP starting_timestamp [ INCLUSIVE | EXCLUSIVE ]
    [ TO TIMESTAMP ending_timestamp [ INCLUSIVE | EXCLUSIVE ] ]
```

Streaming Query

```
SQL
CREATE STREAMING TABLE table_name AS
SELECT select_list
FROM STREAM source_table streaming_cdc_clause [ WITH ( option = value [,
... ] ) ]
[ WHERE condition ]

streaming_cdc_clause:
  CHANGES [ FROM VERSION starting_version [ INCLUSIVE | EXCLUSIVE ] ]
  CHANGES [ FROM TIMESTAMP starting_timestamp [ INCLUSIVE | EXCLUSIVE ] ]
```

Parameters

Parameter	Type	Description
CHANGES FROM ... TO ...	—	Returns changes in range [start, end] (both inclusive by default)
CHANGES FROM ... (no TO)	—	Returns all changes from start to latest version
CHANGES (streaming only)	—	Connector determines starting point (e.g., may emit the full table as INSERTs initially, then stream future changes)
INCLUSIVE	—	Includes the preceding bound in results (this is the default)
EXCLUSIVE	—	Excludes the preceding bound from results; can be applied independently to either or both bounds

Parameter	Type	Description
<code>starting_version</code> , <code>ending_version</code>	LONG or STRING	Version identifiers
<code>starting_timestamp</code> , <code>ending_timestamp</code>	STRING, LONG, or TIMESTAMP	Timestamp values

WITH Options

Option	Type	Default	Description
<code>deduplicationMode</code>	STRING	<code>dropCarryovers</code>	One of: <code>none</code> , <code>dropCarryovers</code> , <code>netChanges</code>
<code>computeUpdates</code>	BOOLEAN	<code>false</code>	When <code>true</code> , derives <code>update_preimage</code> <code>/update_postimage</code> from insert/delete pairs

Output Metadata Columns

Column	Type	Description
<code>_change_type</code>	STRING	<code>insert</code> , <code>delete</code> , <code>update_preimage</code> , or <code>update_postimage</code>
<code>_commit_version</code>	Connector-defined (e.g., LONG)	Version containing this change
<code>_commit_timestamp</code>	TIMESTAMP	Timestamp of the commit

A.2 DataFrame API Changes

New method on `DataFrameReader` and `DataStreamReader`:

```
None
// In org.apache.spark.sql.DataFrameReader
def changes(tableName: String): DataFrame

// In org.apache.spark.sql.streaming.DataStreamReader
```

```
def changes(tableName: String): DataFrame
```

Options are specified via `.option()`:

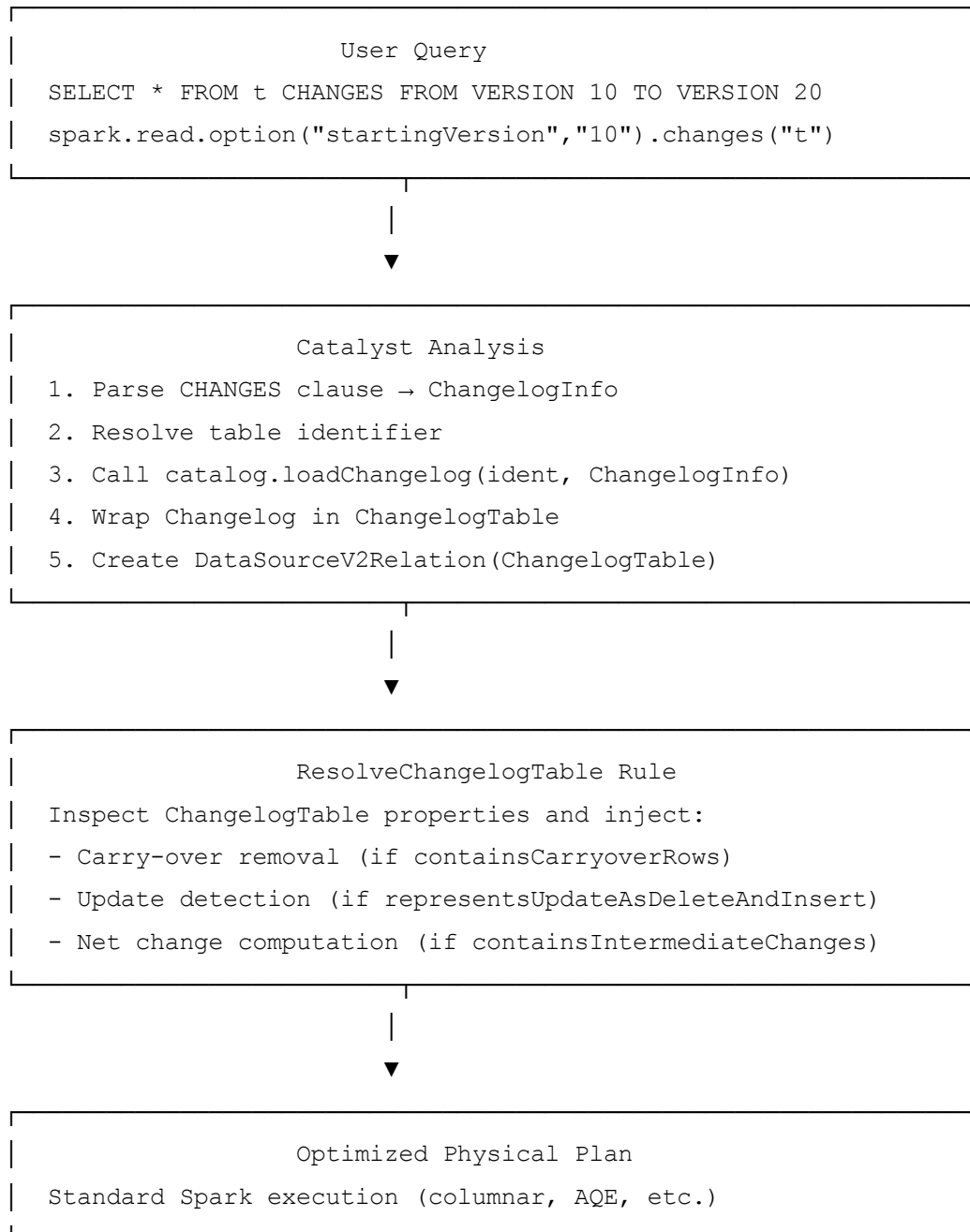
Option	Type	Description
<code>startingVersion</code>	STRING	Start version (inclusive by default)
<code>endingVersion</code>	STRING	End version (inclusive by default); omit for latest
<code>startingTimestamp</code>	STRING	Start timestamp (inclusive by default)
<code>endingTimestamp</code>	STRING	End timestamp (inclusive by default); omit for latest
<code>startingBoundInclusive</code>	BOOLEAN	Include start bound (default: <code>true</code>)
<code>endingBoundInclusive</code>	BOOLEAN	Include end bound (default: <code>true</code>)
<code>deduplicationMode</code>	STRING	<code>none</code> , <code>dropCarryovers</code> (default), or <code>netChanges</code>
<code>computeUpdates</code>	BOOLEAN	Identify update operations (default: <code>false</code>)

A.3 Backward and Forward Compatibility

- **Backward compatible:** No existing APIs are modified or removed. The `CHANGES` keyword is new and does not conflict with existing SQL syntax. `DataFrameReader` and `DataStreamReader` gain a new method; existing methods are unchanged.
 - **Forward compatible:** Connectors that do not implement `loadChangeLog()` will throw `CDC_NOT_SUPPORTED` when users attempt CDC queries. The default implementation in `TableCatalog` throws this error, so no connector is forced to implement CDC.
 - **Connector-specific APIs continue to work:** Delta Lake's `table_changes()`, Iceberg's `.changes`, and similar existing APIs are unaffected.
-

Appendix B. Design Sketch

B.1 Architecture Overview



B.2 Connector API — **Changelog** Interface

The central connector interface is minimal by design. Connectors implement **Changelog** to expose raw change data; Spark handles post-processing.

Java

```
public interface Changelog {
    String name();

    /** Must include _change_type, _commit_version, _commit_timestamp. */
    Column[] columns();

    /** Whether COW produces identical insert/delete pairs. */
    boolean containsCarryoverRows();

    /** Whether multiple intermediate states exist per key. */
    boolean containsIntermediateChanges();

    /** Whether updates are represented as raw delete+insert. */
    boolean representsUpdateAsDeleteAndInsert();

    ScanBuilder newScanBuilder(CaseInsensitiveStringMap options);

    /** Columns uniquely identifying a row (for update detection, net
    changes). */
    default NamedReference[] rowId() { ... }

    /** Column used for ordering in update detection. */
    default NamedReference rowVersion() { ... }
}
```

B.3 TableCatalog Extension

Java

```
// New default method on TableCatalog
default Changelog loadChangelog(Identifier ident, ChangelogInfo
changelogInfo)
    throws NoSuchTableException {
    throw new UnsupportedOperationException("CDC not supported");
}
```

B.4 ChangelogInfo — Query Parameters

Encapsulates the user's CDC query parameters, passed from the parser/DataFrame API to the catalog:

Java

```
public sealed interface ChangelogRange {
    boolean startingBoundInclusive();
}
```

```

    boolean endingBoundInclusive();

    record VersionRange(
        String startingVersion, Optional<String> endingVersion,
        boolean startingBoundInclusive, boolean endingBoundInclusive)
        implements ChangelogRange {}

    record TimestampRange(
        long startingTimestamp, Optional<Long> endingTimestamp,
        boolean startingBoundInclusive, boolean endingBoundInclusive)
        implements ChangelogRange {}

    // No starting/ending boundaries – used by streaming queries
    record Unbounded()
        implements ChangelogRange {
        @Override public boolean startingBoundInclusive() { return true; }
        @Override public boolean endingBoundInclusive() { return true; }
    }
}

public class ChangelogInfo {
    public ChangelogRange range();
    public DeduplicationMode deduplicationMode();
    public boolean computeUpdates();

    public enum DeduplicationMode { NONE, DROP_CARRYOVERS, NET_CHANGES }
}

```

B.5 ChangelogTable — Internal Spark Wrapper

An internal class (not exposed to connectors) that wraps **Changelog** into a **DSv2 Table + SupportsRead**, enabling reuse of **DataSourceV2Relation** without logical plan changes:

```

Java
// Internal – not part of the connector API
public class ChangelogTable implements Table, SupportsRead {
    private final Changelog changelog;
    private final ChangelogInfo changelogInfo;

    @Override public Column[] columns() { return changelog.columns(); }
    @Override public ScanBuilder newScanBuilder(CaseInsensitiveStringMap
options) {
        return changelog.newScanBuilder(options);
    }
    @Override public Set<TableCapability> capabilities() {

```

```

        return Set.of(TableCapability.BATCH_READ,
TableCapability.MICRO_BATCH_READ);
    }
    // ...
}

```

B.6 Analyzer Rule — `ResolveChangelogTable`

The analyzer inspects `DataSourceV2Relation` nodes backed by `ChangelogTable` and conditionally injects post-processing:

```

Java
object ResolveChangelogTable extends Rule[LogicalPlan] {
  override def apply(plan: LogicalPlan): LogicalPlan =
    plan.resolveOperatorsUp {
      case rel @ DataSourceV2Relation(t: ChangelogTable, _, _, _, _) =>
        var result: LogicalPlan = rel
        val cl = t.changelog()
        val info = t.changelogInfo()

        if (info.computeUpdates() && cl.representsUpdateAsDeleteAndInsert())
          result = injectUpdateDetection(result, cl)

        if (info.deduplicationMode() != NONE && cl.containsCarryoverRows())
          result = injectCarryoverRemoval(result, cl)

        if (info.deduplicationMode() == NET_CHANGES &&
            cl.containsIntermediateChanges())
          result = injectNetChangeComputation(result, cl)

        result
    }
}

```

Update Detection Plan

When `computeUpdates() == true` and `representsUpdateAsDeleteAndInsert() == true`, Spark derives `update_preimage/update_postimage` from raw delete/insert pairs.

If carry-over removal is not needed (`containsCarryoverRows() == false` or `deduplicationMode() == NONE`), Spark uses a window function partitioned by `(rowId, rowVersion)`:

```

JSON
Project [data cols, _change_type, _commit_version, _commit_timestamp]
|   (drop intermediate columns _del_cnt, _ins_cnt)
|
└─ Project [*, _change_type (derived)]
    |   _change_type = CASE
    |       WHEN _del_cnt >= 1 AND _ins_cnt >= 1 THEN
    |           IF(_change_type = 'insert', 'update_postimage',
'update_preimage')
    |       ELSE _change_type    -- keep original 'insert' or 'delete'
    |   END
    |
    └─ Window [partitionBy: (rowId, rowVersion)]
        |   _del_cnt = count(if _change_type='delete' then 1 else null)
        |   _ins_cnt = count(if _change_type='insert' then 1 else null)
        └─ DataSourceV2ScanRelation (ChangelogTable)

```

Partitioning by (`rowId`, `rowVersion`) ensures only changes within the same commit are grouped. When both a delete and insert exist in the partition (`_del_cnt >= 1 AND _ins_cnt >= 1`), the row is part of an update and its `_change_type` is rewritten:

- `insert` → `update_postimage`
- `delete` → `update_preimage`

Otherwise, the original `_change_type` is preserved (plain insert or delete). A delete at version 3 and insert at version 5 for the same `rowId` fall in different partitions and remain separate operations.

If carry-over removal is also needed (`containsCarryoverRows() == true` and `deduplicationMode() != NONE`), both are handled together in the carry-over removal plan below, avoiding a separate window function pass.

Carry-Over Removal

When `deduplicationMode() != NONE` and `containsCarryoverRows() == true` (copy-on-write), Spark uses a sort-based approach instead of window functions:

1. **Repartition** by (`rowId`, `rowVersion`) and **sort** by (`rowId`, `rowVersion`, `_change_type ASC`) so each group has at most two rows (one DELETE, one INSERT)
2. **mapPartitions** with a streaming iterator that compares the DELETE+INSERT pair field-by-field:
 - Identical data → carry-over row, drop both
 - Different data → emit as plain `delete/insert` (carry-over removal only)

- Different data → emit as `update_preimage/update_postimage` (if update detection is also requested)

When both carry-over removal and update detection are requested, the iterator handles both in a single shuffle — no need for a separate window function pass.

Net Change Computation

When `deduplicationMode() == NET_CHANGES` and `containsIntermediateChanges() == true`, Spark counts total inserts and deletes per `rowId` and keeps only the net effect (insert if more inserts, delete if more deletes, nothing if equal).

B.7 Streaming Support

Streaming CDC reuses existing DSv2 streaming APIs with no new streaming-specific interfaces:

- `ChangelogTable` reports `MICRO_BATCH_READ` capability
- The connector's `ScanBuilder` returns a `Scan` that implements `toMicroBatchStream()`
- Existing `MicroBatchStream` API handles offsets, partitioning, and rate limiting

Feature	Batch	Streaming
Version range	User-specified	Incremental from checkpoint
Update detection	Supported	Supported (per micro-batch)
Net changes	Supported	Not supported (stateless)

B.8 Deduplication Semantics

Mode	Behavior
<code>none</code>	Raw change rows as-is from the connector
<code>dropCarryovers</code> (default)	Remove identical insert+delete pairs from Copy-On-Write file rewrites
<code>netChanges</code>	Collapse to one net change per row identity

Net change collapse rules:

Change Sequence	Net Result
INSERT → DELETE	No row (cancels out)
INSERT → UPDATE(s)	Final <code>insert</code> only
UPDATE(s)	First pre-image + final post-image
UPDATE(s) → DELETE	<code>delete</code> with original values

B.9 Filter Pushdown Restrictions

To preserve correctness of update detection and deduplication, filter pushdown is restricted:

Column Type	Pushdown Allowed	Reason
Row identity columns	Yes	Same between pre/post-image
<code>_commit_version</code> , <code>_commit_timestamp</code>	Yes	Same for both halves of an update
<code>_change_type</code>	No	Different for pre/post-image
Non-identity data columns	No	May differ between pre/post-image

B.10 Schema Evolution

Schema evolution is the connector's responsibility. Connectors return `InternalRow` or `ColumnarBatch` to Spark and must handle all schema differences across versions internally (type casting, NULL filling, column renames, etc.).

Appendix C. Rejected Designs

Alternative 1: ANSI SQL Temporal Query Syntax

```
SQL
SELECT * FROM table VERSION BETWEEN 10 AND 20;
SELECT * FROM table TIMESTAMP FROM '2024-01-01' TO '2024-01-31';
```

Why rejected:

- **Ambiguity with time travel.** Spark already uses `VERSION AS OF` and `TIMESTAMP AS OF` for point-in-time snapshot queries. Reusing `VERSION` and `TIMESTAMP` keywords for CDC creates confusion — does `VERSION BETWEEN 10 AND 20` return a snapshot or row-level changes?
- **Unclear streaming semantics.** `VERSION FROM x` conflicts with the temporal query pattern and does not naturally extend to streaming.
- **No explicit CDC intent.** Users must infer from the range syntax that they are requesting changes, not snapshots.

Alternative 2: Delta Lake Style Table-Valued Function

SQL

```
SELECT * FROM table_changes('my_table', 10, 20);
```

Why rejected:

- **Table name as string argument.** Passing the table name as a string differs from how tables are typically referenced in Spark SQL, which may complicate catalog resolution, namespace handling, and IDE tooling.
- **Catalog ambiguity.** The TVF must be registered somewhere — but which catalog? This can cause real production issues.

Alternative 3: Virtual Table Pattern (Iceberg Style)

SQL

```
SELECT * FROM my_table.changes WHERE _commit_version BETWEEN 10 AND 20;
```

Why rejected:

- **Namespace collision.** `my_table.changes` could be confused with a nested table or a table named `changes` in the `my_table` schema.
- **Version range in WHERE clause.** The version range looks like a regular row filter, but it actually controls which data files are scanned. This blurs the distinction between CDC query parameters and output predicates.
- **Connector-specific.** The virtual table resolution mechanism is defined by each connector individually, making the syntax and behavior less portable across data sources — a concern that also compounds the namespace collision issue above.